

2026-05-05-p1-f02-permission-rules-design

P1-F02 — Permission Rule System — Design Spec

Date: 2026-05-05 **Author:** Claude Opus 4.7 (1M context) + user (milos85vasic.2nd@gmail.com)
Phase / Feature: Phase 1, Feature 2 of docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md **Status:** APPROVED in brainstorming, awaiting user review of written spec **Successor:** to be handed to superpowers:writing-plans for executable plan **Predecessor:** Feature 1 (Auto-Compaction) — closed 2026-05-05 (commits f0b9b15...d78465c)

1. Goals, non-goals, success criteria

1.1 What we're building

A claude-code-style permission rule system for HelixCode tool execution: every tool call (Read, Edit, Write, Bash, ...) is checked against a layered ruleset that combines a **named mode preset** with **explicit user rules** and **wildcard patterns** (e.g. Bash(`git status`:*)). Compound shell commands (`echo $(rm -rf /)`, `ls && git push`, heredocs, pipelines) are decomposed and **every** leaf call must independently satisfy the rules — closing the smuggling vector that a naive splitter leaves open.

This feature is the substrate that makes the autonomy gradient (`none` → `full_auto`) safe to use in practice: users can leave HelixCode in `semi_auto` and still know that `rm -rf` will always prompt while `git status` never will.

1.2 Goals (priority order)

- **G1 — No bluff.** A passing test/Challenge guarantees end-user usability per Constitution Article XI §11.9. Specifically: a “denied” rule must demonstrably block a real `os/` `exec-issued` command, verified by a filesystem-state diff.
- **G2 — Extend, don't parallelise.** Build on the existing `internal/tools/confirmation/PolicyEngine` (which already has `Policy` → `Rule` → `Condition` with priorities) and the existing `internal/workflow/autonomy/PermissionManager`. Same lesson learned in F01.
- **G3 — Smuggle-resistant compound parsing.** Use `mvdan.cc/sh/v3/syntax` to extract every call expression from a Bash command, including command substitutions, backticks, heredocs, pipelines, and quoted operators.
- **G4 — File-first, no DB.** Rules live in `~/helixcode/permissions.yaml` (user-global) and `<project>/helixcode/permissions.yaml` (project-local). No PostgreSQL, no Redis. Tests stay cheap, users can `grep/diff` their rules.
- **G5 — Discoverable CLI.** Both a Cobra subcommand group (`helixcode permissions {list,add,remove,check}`) and an in-session slash command (`/permissions ...`) over the existing `internal/commands/dispatcher`.

1.3 Non-goals (explicit out-of-scope for F02)

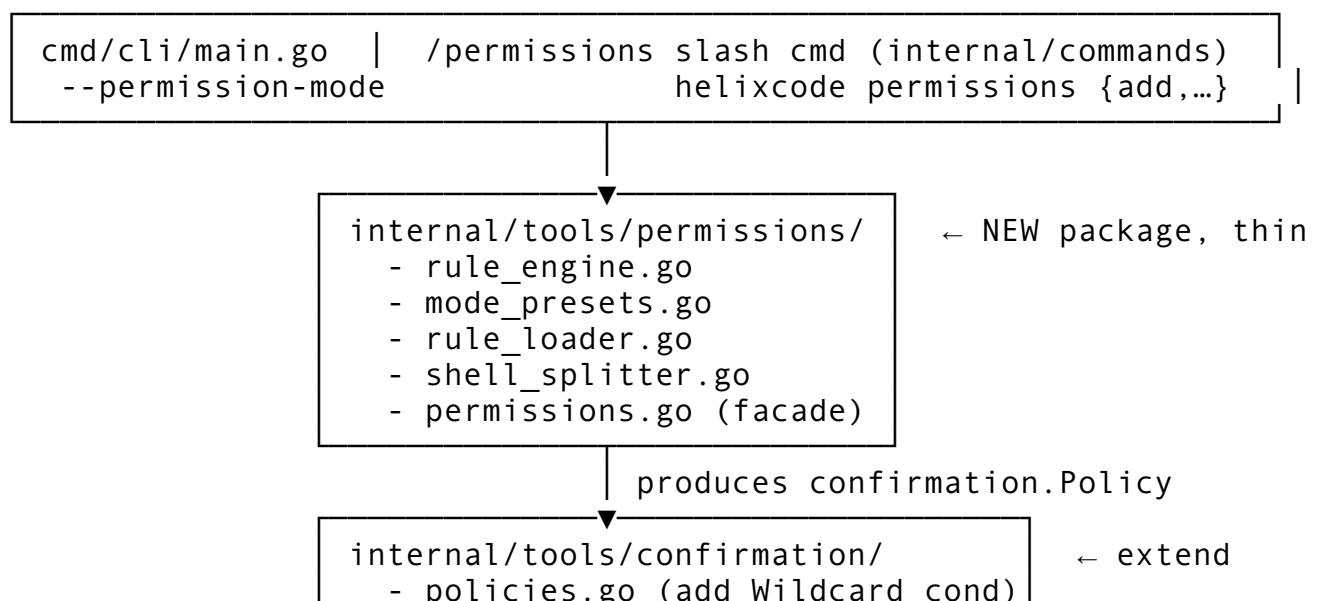
- **N1.** Database-backed rule store. Deferred; the file-only design is sufficient until team/multi-tenant scenarios appear.
- **N2.** REST API for rule management. No consumer.
- **N3.** Per-MCP-server rule scoping. Belongs to F06 (MCP Full Lifecycle).
- **N4.** Replacing `AutonomyMode`. The five claude-code permission modes become **named rule presets** that compose with the existing autonomy gradient — not a parallel mode taxonomy.
- **N5.** Rule import/export, rule profiles, organisation-level policies. Defer until requested.

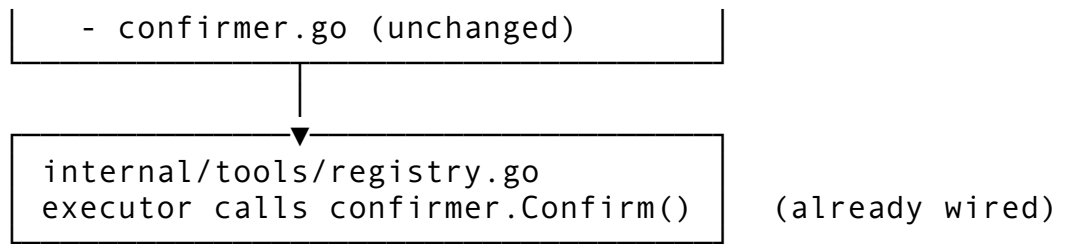
1.4 Success criteria

- **S1.** `make verify-compile` exits 0 with the new package and all consumers wired.
- **S2.** Unit tests for the new package and the extended `confirmation.Condition` pass with `-race`.
- **S3.** Integration test (`-tags=integration`, no mocks) demonstrates that a `Bash(rm -rf *)` deny rule blocks a real `os/exec` invocation against a temp filesystem.
- **S4.** Challenge under `tests/e2e/challenges/permissions/` passes three end-to-end scenarios (read auto-allow, destructive deny under `--permission-mode auto`, command-substitution smuggle denied) with stdout transcript + filesystem-state diff pasted into the close-out commit body.
- **S5.** Anti-bluff smoke (`grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/tools/permissions/`) returns zero hits.
- **S6.** `helixcode --permission-mode <preset>` and `helixcode permissions {list,add,remove,check}` work end-to-end against a real filesystem.
- **S7.** `/permissions` slash command registered and dispatched via existing `internal/commands/`.

2. Architecture

2.1 Topology





2.2 Why this shape

- **One new package, narrow responsibility.** `internal/tools/permissions/` only loads/parses/dispatches; the actual ask-or-not logic stays in the existing `confirmation.Confirmer`.
- **One field added to existing types.** `confirmation.Condition` gets a `Wildcard` string field; `Condition.Matches` is extended to evaluate it. No other package signature changes.
- **No new mode enum.** The 5 claude-code modes are package-private rule slices in `mode_presets.go`. The user-facing `AutonomyMode` (`none`, `basic`, `basic_plus`, `semi_auto`, `full_auto`) is unchanged.
- **No new persistence layer.** Files only. The existing audit log (`internal/tools/confirmation/audit.go`) records rule matches.

2.3 Component responsibilities

Component	Responsibility
<code>rule_engine.go</code>	parse rule patterns, sort by priority, evaluate against a <code>ToolCallRequest</code> , resolve to <code>confirmation.Action</code>
<code>mode_presets.go</code>	the five built-in <code>[]Rule</code> slices (<code>default</code> , <code>auto</code> , <code>acceptEdits</code> , <code>dontAsk</code> , <code>bypassPermissions</code>) and the conservative read-only / write command lists
<code>rule_loader.go</code>	load YAML from user + project paths, fail-fast on malformed YAML or unknown preset, produce a <code>RuleSet</code> with <code>Source</code> provenance
<code>shell_splitter.go</code>	<code>mvdan.cc/sh/v3/syntax</code> walker that extracts every call expression from a Bash string, recursing into command substitutions and backticks
<code>permissions.go</code>	facade: <code>NewEngine(loader, mode)</code> returns an engine that registers <code>confirmation.Policys</code> with the existing <code>PolicyEngine</code>
<code>confirmation/policies.go</code> (extended)	add <code>Wildcard</code> string to <code>Condition</code> ; <code>Matches</code> evaluates it against <code>req.Parameters["command"]</code> (or the relevant arg) using <code>shell_splitter</code>
<code>cmd/cli/main.go</code> (extended)	

Component	Responsibility
	parse <code>--permission-mode</code> flag; on startup, load files and register policy with the confirmer
helixcode permissions Cobra group	subcommands <code>list</code> , <code>add</code> , <code>remove</code> , <code>check</code> ; writes/reads YAML files
<code>/permissions</code> slash command	registers a <code>commands.Command</code> that dispatches to the same Cobra runners; updates the in-memory rule engine for the rest of the session

3. Data shapes

3.1 YAML on disk

```
# ~/.helixcode/permissions.yaml (user-global)
# <project>/helixcode/permissions.yaml (project-local; takes
# precedence on conflict)
mode:
    acceptEdits          # one of: default | auto |
    acceptEdits | dontAsk | bypassPermissions
rules:
  - pattern: "Bash(git status:*)"
    action: allow         # allow | ask | deny
    priority: 100         # higher wins; default 0
    description: "git status is read-only"
  - pattern: "Bash(rm -rf *)"
    action: deny
    priority: 1000
  - pattern: "Edit(internal/secrets/*)"
    action: ask
```

3.2 Pattern grammar

```
pattern ::= ToolName "(" arg-pattern ")"
ToolName ::= [A-Za-z][A-Za-z0-9_]*
arg-pattern ::= glob (with `*`, `?`, `[abc]`)
```

Examples: `Read(*.go)`, `Write(internal/auth/*)`, `Bash(git status:*)`, `Edit(*.md)`.

For Bash, the pattern's `arg-pattern` matches the command **after** compound splitting; `Bash(git status:*)` allows every leaf call whose normalised string starts with `git status`. The `:` separator after the verb is preserved from the porting doc for compatibility with documented claude-code patterns.

3.3 Go types (excerpt)

```
// internal/tools/permissions/rule_engine.go
type Rule struct {
    Pattern      string           // "Bash(git status:*)"
    Action      confirmation.Action // ActionAllow /
                ActionAsk / ActionDeny
    Priority     int
    Description  string
    Source       string           // "project" | "user" |
                "preset" | "cli"
}

type RuleSet struct {
    Mode      string // "acceptEdits" etc.
    Rules     []Rule
    Sources   []string // resolved file paths in load order
}

type FileLoader struct {
    UserPath      string
    ProjectPath   string
}

func (l *FileLoader) Load(ctx context.Context) (*RuleSet, error)
func (l *FileLoader) Save(ctx context.Context, scope Scope, r
    Rule) error

type Scope int
const (
    ScopeUser Scope = iota
    ScopeProject
    ScopeSession
)
```

3.4 Resolution order

Layer order (earliest layer wins on identical patterns): 1. CLI flag (session) 2. Project file (<project>/.`helixcode/permissions.yaml`) 3. User file (~/.`helixcode/permissions.yaml`) 4. Preset's built-in rules (selected by mode: or `--permission-mode`) 5. Default action — `ActionAsk`

Identical-pattern rule: if two layers contain the *same* pattern string, the earlier layer's rule replaces the later layer's rule (priority is irrelevant to the override). Different patterns from different layers all coexist in the resolved set.

Within the resolved set: evaluation is priority-sorted (highest first); the first rule whose pattern matches the request wins.

The resolved `RuleSet` is materialised as a single `confirmation.Policy` registered with the `PolicyEngine` at engine construction time and on every reload.

4. Mode preset semantics

Each preset is a fixed []Rule slice in mode_presets.go:

Preset	Built-in rules
default	(no rules) → falls through to ActionAsk for every tool call
auto	*(*) → allow at priority 0 (overridden by any explicit user deny)
acceptEdits	Edit(*) → allow, Write(*) → allow, MultiEdit(*) → allow, Bash(<write commands>) → allow
dontAsk	Read(*) → allow, Glob(*) → allow, Grep(*) → allow, Bash(<read-only commands>) → allow
bypassPermissions	*(*) → allow at priority 1_000_000 (highest); operator-only safety hatch

<read-only commands> and <write commands> are conservative package-private lists derived from the porting doc. Examples: - read-only: ls, cat, find, grep, git status, git log, git diff, pwd, echo, head, tail, wc, ps, env, which, whoami, date, go version, node --version, ... - write: git add, git commit, git push, git checkout, git reset, rm, mv, cp, mkdir, chmod, chown, tar, wget, curl, make, docker, kubectl, ...

A command is auto-allowed only if its **leaf call** starts with one of the listed prefixes. We err on the side of asking — every preset that auto-allows can be overridden by an explicit deny rule.

5. Compound-command splitting (G3)

```
// internal/tools/permissions/shell_splitter.go
func SplitCommands(input string) ([]string, error) {
    parser := syntax.NewParser()
    f, err := parser.Parse(strings.NewReader(input), "")
    if err != nil {
        return nil, err
    }
    var cmds []string
    syntax.Walk(f, func(n syntax.Node) bool {
        if call, ok := n.(*syntax.CallExpr); ok &&
            len(call.Args) > 0 {
            cmds = append(cmds, syntax.NodeString(call))
        }
        return true
    })
}
```

```
    return cmds, nil
}
```

Wildcard match runs against **every** extracted call expression independently. Each leaf call resolves to one of `allow` / `ask` / `deny` via the normal evaluation pipeline (§3.4).

Aggregation rule (compound → single decision): - if **any** leaf resolves to `deny` → compound is **denied** (most-restrictive wins) - else if **any** leaf resolves to `ask` → compound is **asked** (single prompt that lists all leaves) - else (every leaf resolved to `allow`) → compound is **allowed**

Command substitutions (`$(...)`, backticks) are walked recursively by `syntax.Walk` — `echo $(rm -rf /)` extracts `echo` and `rm -rf /` as two separate call expressions, both of which independently feed into the aggregation. This closes the smuggling vector that the porting doc’s simple splitter leaves open.

A shell parser error (malformed input) **denies** the request (fail-closed) and emits an audit-log event. We never silently drop a malformed input on the assumption that “we couldn’t parse it, so it must be safe.”

6. CLI surface

6.1 Startup flag

```
helixcode --permission-mode <preset> ...
```

`<preset>` is one of `default`, `auto`, `acceptEdits`, `dontAsk`, `bypassPermissions`. Unknown values fail fast with the list of valid names. The flag overrides the `mode`: key in YAML files for the current session only.

6.2 Cobra subcommand group

```
helixcode permissions list [--scope all|user|project|preset]
helixcode permissions add "Bash(git status:*)" allow [--scope user|project]
helixcode permissions remove "Bash(git status:*)" [--scope user|project]
helixcode permissions check Bash --command "git status -sb"
```

`check` is a dry-run: it runs the full evaluation against the current ruleset and prints the matched rule + decision **without invoking any tool**.

6.3 In-session slash command

```
/permissions                                # show effective rules (same as
/permissions mode acceptEdits                # change mode for the rest of
/permissions add Bash(git push:*) ask
/permissions remove Bash(git push:*)
```

Registered with `internal/commands/` (existing dispatcher; F09 will standardise the user-facing slash-command experience further). Mutations in this scope are session-only — they do **not** write to disk unless the user runs the equivalent `helixcode permissions add` form.

7. Error handling and edge cases

Case	Behaviour
Malformed YAML in user file	startup fails fast with line:col error pointer; never silently fall back
Malformed pattern (unparseable)	rule is rejected at load; logged with file:line; engine continues with valid rules
Unknown preset name	startup fails fast with the list of valid names
Shell parser error on input	request is denied (fail-closed); event emitted to audit log
File missing	treated as empty ruleset (not an error)
Conflicting rules at same priority	last-loaded wins; warning logged once at startup
User passes <code>--permission-mode bypassPermissions</code>	log a WARN-level event with timestamp + session ID; record in audit log; require no extra confirmation (operator-explicit)
<code>permissions add</code> writes to a missing config dir	dir auto-created with mode 0700; file mode 0600 (matches <code>.env</code> precedent in CONST-042)

Audit logging reuses `internal/tools/confirmation/audit.go`. Every rule match and every fallback decision is recorded with: timestamp, tool name, raw input, matched rule (or default), action taken, scope source.

8. Testing strategy (CONST-035 / Article XI §11.9)

Three layers, mirroring F01:

8.1 Unit (`internal/tools/permissions/*_test.go`)

Table tests covering: - Pattern parsing (valid, malformed, edge cases like empty arg-pattern). - Wildcard matching (`*`, `?`, `[abc]`, escaped chars, anchored vs unanchored). - Compound splitting: `&&`, `|`, `;`, `|`, `$(...)`, backticks, heredocs, quoted operators (`echo "foo && bar" → 1 leaf, not 2`), escaped operators, multi-line scripts. - Mode-preset rule generation: each preset produces the documented rule set. - File loader: missing file = empty, malformed YAML = error, unknown preset = error, project overrides user, source provenance preserved. - Rule engine Evaluate: priority sort, allow/deny/ask resolution, fall-through to default.

Mocks are allowed at this layer.

8.2 Integration (`tests/integration/permissions/permissions_integration_test.go`, `-tags=integration`)

Runs the real `confirmation.Confirmer` + real file loader against a temp `XDG_CONFIG_HOME`. Asserts: - A `Bash(rm -rf *)` deny rule blocks a real tool registry call (the target file is **not** deleted). - A `Bash(git status:*)` allow rule lets `git status`

execute and return real output. - The smuggle case: Bash(echo *) allow + a destructive substitution → denied with audit-log entry.

No mocks at this layer (per CLAUDE.md / Constitution Rule 5).

8.3 Challenge (tests/e2e/challenges/permissions/)

Full CLI invocation under the docker-compose-full-test stack. expected.json lists three scenarios:

- 1. **Read auto-allowed under** dontAsk.helixcode --permission-mode dontAsk -p "list files" runs ls without prompting; transcript captured.
- 2. **Destructive denied under** default.helixcode --permission-mode default -p "delete the marker" is asked to confirm; the test rejects; the marker file is untouched (verified by stat).
- 3. **Smuggle denied.** helixcode --permission-mode auto -p "echo 'hello' && rm -rf /tmp/marker" is denied because rm -rf matches the deny rule; /tmp/marker is untouched (verified by stat).

Runtime evidence: stdout transcript + stat /tmp/marker output pasted into the close-out commit body.

8.4 Anti-bluff smoke

grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" internal/tools/permissions/ must be empty. Run on every sub-commit.

8.5 Mutation test (CONST-039)

Deliberately remove the deny-rule check from the engine. The Challenge MUST fail. If it still passes, the Challenge is non-conformant and must be tightened.

9. Sub-task plan

Mirroring F01’s 11-task cadence (here 13 because of the additional CLI surface and slash-command wiring):

#	Task	Outputs
T01	Bootstrap evidence file 06_phase_1_evidence.md §F02 + advance PROGRESS to F02-active	docs only
T02	Add Wildcard field to confirmation.Condition; extend Matches (TDD)	unit tests pass
T03	New package internal/tools/permissions skeleton (types only, doc.go)	compile-only
T04		go.mod bump committed

#	Task	Outputs
	shell_splitter.go + mvdan.cc/sh/v3/syntax dep + TDD	
T05	rule_engine.go (pattern parse, match, priority sort) — TDD	unit tests pass
T06	mode_presets.go — five presets + conservative command lists + tests	unit tests pass
T07	rule_loader.go (YAML → Rule, file precedence, fail-fast errors) — TDD	unit tests pass
T08	permissions.go facade: NewEngine builds confirmation.Policy and registers with PolicyEngine	unit tests pass
T09	Wire --permission-mode flag in cmd/cli/main.go; integration test (no mocks, real files)	-tags=integration test passes
T10	Add helixcode permissions {list,add,remove,check} subcommands (Cobra)	smoke test passes
T11	Register /permissions slash command in internal/ commands/	unit + smoke test
T12	Challenge under tests/e2e/ challenges/permissions/ with three scenarios from §8.3; runtime evidence pasted	Challenge PASS in commit body
T13	Feature 2 close-out: anti-bluff scan, make verify- foundation, push to all four remotes (no force)	PROGRESS.md flipped to F03 active

Estimated 13 sub-commits. F03 (Tool Result Persistence) is unblocked when T13 lands.

10. Risks and mitigations

Risk	Mitigation
mvdan.cc/sh/v3 transitively pulls a heavy dep	Verify before T04 commit: go mod why mvdan.cc/sh/v3 and go mod download size. If it brings in something unwanted, fall back to a smaller subset

Risk	Mitigation
	(<code>mvdan.cc/sh/v3/syntax</code> only, which is the core parser without the executor).
Existing <code>autonomy.PermissionManager</code> callers expect old semantics	The autonomy package is not modified by F02 — both systems coexist; tool-level permission is added below the autonomy layer. Confirmed by greppable callsite audit at T03.
Challenge needs Anthropic API key for end-to-end run	<code>tests/e2e/challenges/permissions/</code> uses a deterministic prompt + the existing test infrastructure (already validated for F01-T10). Same <code>.env.full-test</code> Anthropic key.
YAML schema drift	Schema is version-locked: top-level <code>apiVersion:</code> <code>helixcode.permissions/v1</code> becomes mandatory in T07; loader rejects unknown versions.
CLI flag conflicts with existing flags	<code>--permission-mode</code> audit at T09: <code>grep -rn 'permission-mode\ PermissionMode' cmd/</code> confirms no collision before adding.

11. References

- Synthesis spec: `docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md` §4.1 (Phase 1 charter)
- Porting doc: `docs/improvements/04_main_plan_step_02/kimi_agent_helix_cli_integration_blueprint/porting_claude_code.md` §Feature 2
- Predecessor plan: `docs/superpowers/plans/2026-05-05-p1-f01-auto-compaction.md`
- Evidence file (live): `docs/improvements/06_phase_1_evidence.md`
- Existing infrastructure being extended:
 - `helix_code/internal/tools/confirmation/policies.go` — `PolicyEngine, Policy, Rule, Condition`
 - `helix_code/internal/tools/confirmation/audit.go` — audit log
 - `helix_code/internal/workflow/autonomy/permission.go` — autonomy-level `PermissionManager` (unchanged)
 - `helix_code/internal/commands/command.go` — slash-command interface
- Constitutional anchors:
 - Article XI §11.9 — Anti-Bluff Forensic Anchor (every PASS carries runtime evidence)
 - CONST-035 — Zero-Bluff Mandate
 - CONST-039 — Challenge System Integrity (mutation testing mandatory)
 - CONST-042 — No-Secret-Leak (file mode 0600 for any persisted YAML containing secrets — N/A here, but the dir-creation pattern matches)
 - CONST-043 — No-Force-Push (close-out commit T13 pushes without force)

End of P1-F02 Permission Rule System design spec.